

Arithmetic



ARITHMETIC

Write a Java program that takes two integers as input from the user and calculates their sum, difference, product, and quotient using arithmetic operators.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class first
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int a=sc.nextInt();
8         int b=sc.nextInt();
9         int c=a+b;
10        int diff=a-b;
11        int pro=a*b;
12        int quo=a%b;
13        System.out.println("Welcome to simats");
14        System.out.println("Addition of two number:"+ c);
15        System.out.println("diff of two number:"+ diff);
16        System.out.println("product of two number:"+ pro);
17        System.out.println("quotient of two number:"+ quo);
18    }
19 }
```

OUTPUT

```
Welcome to simats
Addition of two number:5
diff of two number:-1
product of two number:6
quotient of two number:2
```

INPUT

```
2
3
```

MAX



MAX

Write a Java program to find the maximum of three numbers using conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class first
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int a=sc.nextInt();
8         int b=sc.nextInt();
9         int temp=a;
10        a=b;
11        b=temp;
12        System.out.println(" a = "+ a +" , b = "+b);
13    }
14 }
```

OUTPUT

```
a = 5 , b = 4
```

INPUT

4

5

Leap



LEAP

Write a Java program to check whether a given year is a leap year or not using conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class first
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int a=sc.nextInt();
8         if(a%4==0||a%400==0 && a%100!=0)
9         {
10             System.out.println("A is a leap year"+ a);
11         }
12         else
13         {
14             System.out.println("A is not a leap year"+ a);
15         }
16     }
17 }
```

OUTPUT

```
A is not a leap year2005
```

INPUT

2004

Fibonacci



FIBONACCI

Write a Java program to print the Fibonacci series up to a given number using loops.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class fibonacci
3 {
4     public static void main(String[] arg)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.print("enter the limit:");
8         int limit=sc.nextInt();
9         int first=0 ,second=1;
10        System.out.println("Fibonacci series:"+ limit);
11        while (first<=limit){
12            System.out.print(first +" ");
13            int next=first+second;
14            first=second;
15            second=next;
16        }
17    }
18 }
```

OUTPUT

```
enter the limit:Fibonacci series:20
0 1 1 2 3 5 8 13
```

INPUT

20

Factorial



FACTORIAL

Write a Java program to calculate the factorial of a given number using loops.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class factorial{
3     public static void main(String[] args){
4         Scanner sc =new Scanner(System.in);
5         System.out.println("enter a number: ");
6         int n=sc.nextInt();
7         long factorial = 1;
8         for (int i = 1; i <= n; i++){
9             factorial *=i;
10        }
11        System.out.println("factorial of"+ n +" = "+factorial);
12        sc.close();
13    }
14 }
```

OUTPUT

```
enter a number:
factorial of5 = 120
```

INPUT

5

RT-Pattern

RT-PATTERN

Write a Java program to print the following pattern using loops:

```
**
***
****
*****
```

PROGRAM EDITOR

Java



Run

Save

```
1 public class starpattern{
2     public static void main(String[] args){
3         for(int i=1;i<=5;i++){
4             for(int j=1;j<=i;j++){
5                 System.out.print("*");
6             }
7             System.out.println();
8         }
9     }
10 }
```

INPUT

Your INPUT here!

OUTPUT

```
*
**
***
****
*****
```

Search



SEARCH

Write a Java program to implement binary search on an array of integers using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class binarysearch
3 {
4     public static void main(String[] args)
5     {
6         int arr[]={10,20,30,40,50,60};
7         Scanner sc=new Scanner(System.in);
8         System.out.println("enter the number to search:");
9         int key=sc.nextInt();
10        int low=0;
11        int high=arr.length-1;
12        int position=-1;
13        while(low<=high)
14        {
15            int mid=(low+high)/2;
16            if(arr[mid]==key)
17            {
18                position=mid;
19                break;
20            }
21            else if(arr[mid]<key)
```

OUTPUT

```
enter the number to search:
element found at index: 0
```

INPUT

10

GCD

GCD

Write a Java program to find the GCD (Greatest Common Divisor) of two numbers using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class GCD{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         System.out.println("enter the first integer:");
6         int a=sc.nextInt();
7         System.out.println("enter the second integer:");
8         int b=sc.nextInt();
9         while(b!=0){
10             int temp=b;
11             b=a%b;
12             a=temp;
13         }
14         System.out.print("GCD= "+ a);
15     }
16 }
```

OUTPUT

```
enter the first integer:
enter the second integer:
GCD= 6
```

INPUT

```
30
18
```


Prime



PRIME

Write a Java program to check whether a given number is a prime number or not using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class prime{
3     public static void main(String[] args)
4     {
5         Scanner sc=new Scanner(System.in);
6         System.out.println("Enter a number: ");
7         int n=sc.nextInt();
8         int count=0;
9         for(int i=1;i<=n;i++)
10        {
11            if(n%i==0)
12            {
13                count++;
14            }
15        }
16
17        System.out.print(n +"is a prime number");
18    }
19    else{
20        System.out.print(n +"is not a prime number");
21    }
```

OUTPUT

```
Enter a number:
5is a prime number
```

INPUT

5

Sort



SORT

Write a Java program to implement selection sort on an array of integers using loops and conditional statements.

```
1 import java.util.Scanner;
2 public class selectionsort
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the number of elements:");
8         int n=sc.nextInt();
9         int [] arr=new int [n];
10        System.out.println("enter the array element:");
11        for(int i=0;i<n;i++){
12            arr[i]=sc.nextInt();
13        }
14        for(int i=0;i<n-1;i++)
15        {
16            int min=i;
17            for(int j=i+1;j<n;j++)
18            {
19                if(arr[j]<arr[min])
20                {
21                    min=j;
```

OUTPUT

```
enter the number of elements:
enter the array element:
sorted array:
1
3
5
6
7
```

5

5 7 3 1 6

TABLE

Write a Java program to print the multiplication table of a given number using loops.

```
1 import java.util.Scanner;
2 public class multiplicationtable
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int n=sc.nextInt();
8         for(int i=1;i<=10;i++){
9             System.out.println(n + "x" + i +"=" + (n * i));
10        }
11        sc.close();
12    }
13 }
```

OUTPUT

```
7x1=7
7x2=14
7x3=21
7x4=28
7x5=35
7x6=42
7x7=49
7x8=56
```

Factorial



FACTORIAL

2. Write a Java program to find the factorial of a given number using recursion.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class factorial{
3     static int factorial(int n){
4         if(n==0 || n==1){
5             return 1;
6         }
7         else{
8             return n* factorial(n-1);
9         }
10    }
11    public static void main(String[] args){
12        Scanner sc=new Scanner(System.in);
13        System.out.println("enter a number");
14        int num=sc.nextInt();
15        System.out.println("factorial = "+ factorial(num));
16    }
17 }
```

OUTPUT

```
enter a number
factorial = 120
```

INPUT

5

LT-Pattern

LT-PATTERN

Write a Java program to print the following pattern using loops:*****

**

*

PROGRAM EDITOR

Java



Run

Save

```
1 public class LT{
2     public static void main(String[] args){
3         for(int i=5;i>=1;i--){
4             for(int j=1;j<=i-1;j++){
5                 System.out.print("*");
6             }
7             System.out.println();
8         }
9     }
10 }
```

OUTPUT

```
*****
****
***
**
*
```

INPUT

Your INPUT goes here!

LCM



LCM

Write a Java program to find the LCM (Least Common Multiple) of two numbers using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class LCM
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int a=sc.nextInt();
8         int b=sc.nextInt();
9         int x=a, y=b;
10        while(y!=0)
11        {
12            int temp=y;
13            y=x%y;
14            x=temp;
15        }
16        int lcm=(a*b)/x;
17        System.out.print("LCM="+lcm);
18    }
19 }
```

OUTPUT

```
LCM=30
```

INPUT

```
6
5
```

D to B



D TO B

Write a Java program to convert a decimal number to binary using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class decimaltobinary
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the decimal value:");
8         int decimal=sc.nextInt();
9         if(decimal==0){
10             System.out.println("Binary:0");
11         }
12         else
13         {
14             int[] binary=new int[32];
15             int i=0;
16             while(decimal>0)
17             {
18                 binary[i]=decimal%2;
19                 decimal=decimal/2;
20                 i++;
21             }
```

OUTPUT

```
enter the decimal value:
binary: 11001
```

INPUT

25

B to D

B TO D

Write a Java program to convert a binary number to decimal using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 public class matrixaddition
2 {
3     public static void main(String[] args)
4     {
5         int m1[][]={{1,2},{3,4}};
6         int m2[][]={{5,6},{7,8}};
7         int rows=2;
8         int cols=2;
9         int m3[][]=new int[rows][cols];
10        for(int i=0;i<rows;i++)
11        {
12            for(int j=0;j<cols;j++)
13            {
14                m3[i][j]=m1[i][j]*m2[i][j];
15                System.out.print(m3[i][j]+"");
16            }
17        }
18        System.out.println();
19    }
20 }
21 }
```

OUTPUT

```
sum of two matrix:
5 12
21 32
```

INPUT

Your INPUT goes here!

Sum



SUM

Write a Java program to calculate the sum of the digits of a given number using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

INPUT

12345

```
1 import java.util.Scanner;
2 public class sumofdigits
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int n=sc.nextInt();
8         int sum=0;
9         while(n!=0)
10        {
11            sum+=n%10;
12            n/=10;
13        }
14        System.out.print("Sum of the digits="+sum);
15    }
16 }
```

OUTPUT

```
Sum of the digits=15
```

Product



PRODUCT

Write a Java program to calculate the product of the digits of a given number using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class product
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int n=sc.nextInt();
8         int product=1;
9         while(n!=0)
10        {
11            product=product*(n%10);
12            n=n/10;
13        }
14        System.out.print("product="+product);
15    }
16 }
```

OUTPUT

```
product=24
```

INPUT

1234

Power



POWER

Write a Java program to find the power of a number using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class hallowpattern
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the number of rows:");
8         int rows=sc.nextInt();
9         System.out.println("enter the no of cols:");
10        int cols=sc.nextInt();
11        for(int i=1;i<=rows;i++){
12            for(int j=1;j<=cols;j++){
13                |
14                System.out.print("*");
15
16                System.out.print("");
17
18            }
19            System.out.println();
20        }
21        sc.close();
```

OUTPUT

```
enter the number of rows:
enter the no of cols:
* * * * *
* * * * *
* * * * *
* * * * *
```

INPUT

```
5
5
```

Area Circle



AREA CIRCLE

Write a Java program to calculate the area of a circle using the radius input by the user.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;  
2 public class areaofcircle  
3 {  
4     public static void main(String[] args)  
5     {  
6         Scanner sc=new Scanner(System.in);  
7         int r=sc.nextInt();  
8         double area=3.14*(r*r);  
9         System.out.print("area of the circle"+ area);  
10    }  
11 }
```

OUTPUT

```
area of the circle153.86
```

INPUT

7

Shpere



SHPERE

Write a Java program to calculate the volume of a sphere using the radius input by the user.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class volume
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int r=sc.nextInt();
8         double volume=(4/3)*3.14*(r*r*r);
9         System.out.print("volume of sphere:"+ volume);
10    }
11 }
```

OUTPUT

```
volume of sphere:392.5
```

INPUT

5

Palindrome



PALINDROME

Write a Java program to check whether a given string is a palindrome or not using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class palindromeString
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter a string:");
8         String str = sc.nextLine();
9         String rev= "";
10        for(int i = str.length() - 1; i >= 0; i--)
11        {
12            rev = rev + str.charAt(i);
13        }
14        if(str.equals(rev))
15        {
16            System.out.print("palindrome");
17        }
18        else
19        {
20            System.out.print("not a palindrome");
21        }
```

OUTPUT

```
enter a string:
palindrome
```

INPUT

madam

Armstrong

ARMSTRONG

Write a Java program to check whether a given number is an Armstrong number or not using loops and conditional statements.

```
1 import java.util.Scanner;
2 public class pyramid
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the number of rows");
8         int rows=sc.nextInt();
9         for(int i=1;i<=rows;i++){
10             for(int j=1;j<=rows-i;j++){
11                 System.out.print(" ");
12             }
13             for(int k=1;k<=(2*i-1);k++){
14                 |
15                 System.out.print("*");
16
17                 System.out.print("");
18
19             }
20             System.out.println();
21 }
```

OUTPUT

```
enter the number of rows
    *
  ***
*****
*****
*****
*****
```

5

RT-num.Pattern ▾

RT- NUM.PATTERN

Write a Java program to
print the following
pattern using loops:1

```
22
333
4444
55555
```

PROGRAM EDITOR

Java ▾

Run

Save

INPUT

```
1 import java.util.Scanner;
2 public class numberpattern{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         System.out.println("enter the no of rows:");
6         int rows=sc.nextInt();
7         for(int i=1;i<=rows;i++){
8             for(int j=1;j<=i;j++){
9                 System.out.print(i);
10            }
11            System.out.println();
12        }
13    }
14 }
```

5

OUTPUT

```
enter the no of rows:
22
333
4444
55555
```


Perfect

**PERFECT**

Write a Java program to check whether a given number is a perfect number or not using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class perfectnumber
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int a=sc.nextInt();
8         int sum=0;
9         for(int i=1;i<a;i++)
10        {
11            if(a%i==0)
12                sum+=i;
13        }
14        if(a==sum)
15            System.out.print("perfect number");
16        else
17            System.out.print("not a perfect number");
18    }
19 }
```

OUTPUT

```
perfect number
```

INPUT

6

Max Array

MAX ARRAY

Write a Java program to find the largest element in an array of integers using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```

1 import java.util.Scanner;
2 public class rectangle
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the number of row :");
8         int rows= sc.nextInt();
9         System.out.println("enter the number of column:");
10        int cols = sc.nextInt();
11        for(int i=1;i<=rows;i++)
12        {
13            for(int j=1;j<=cols;j++)
14            {
15                System.out.print("*");
16            }
17            System.out.println(" ");
18        }
19    }
20 }
```

OUTPUT

```

enter the number of row :
enter the number of column:
* * * * *
* * * * *
* * * * *
```

INPUT

3
5

2nd Max



2ND MAX

Write a Java program to find the second largest element in an array of integers using loops and conditional statements.

```
1 import java.util.Scanner;
2 public class pyramid
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("Enter no of rows:");
8         int rows=sc.nextInt();
9         for(int i=1;i<=rows;i++)
10        {
11            for(int j=rows;j>i;j--)
12            {
13                System.out.print(" ");
14            }
15            for(int k=1;k<=(2*i-1);k++)
16            {
17                System.out.print("*");
18            }
19            System.out.println();
20        }
21        for(int i=rows-1;i>=1;i--)
```

OUTPUT

```
Enter no of rows:
```

```
  *
 ***
*****
*****
*****
*****
*****
```

5

Min



MIN

Write a Java program to find the smallest element in an array of integers using loops and conditional statements.

```
1 import java.util.Scanner;
2 public class minimum
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the number of element:");
8         int n=sc.nextInt();
9         int[] arr=new int[n];
10        System.out.println("enter the array element:");
11        for(int i=0;i<n;i++)
12        {
13            arr[i]=sc.nextInt();
14        }
15        int smallest=arr[0];
16        for(int i=1;i<n;i++){
17            if(arr[i]<smallest){
18                smallest=arr[i];
19            }
20        }
21        System.out.println("the smallest element is:"+smallest);
```

OUTPUT

```
enter the number of element:
enter the array element:
the smallest element is:2
```

```
5
8
9
3
6
2
```

2nd Min



2ND MIN

Write a Java program to find the second smallest element in an array of integers using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class secondsmallest
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the number of element:");
8         int n=sc.nextInt();
9         int [] a=new int[n];
10        System.out.println("enter the array elements:");
11        for(int i=0;i<n;i++)
12        {
13            a[i]=sc.nextInt();
14        }
15        int smallest=a[0];
16        int secondSmallest=a[0];
17        for(int i=1;i<a.length;i++)
18        {
19            if(a[i]<smallest)
20            {
21                smallest=a[i];
```

OUTPUT

```
enter the number of element:
enter the array elements:
Second smallest element=4
```

INPUT

```
5
4 5 7 2 8
```

Sort-B

SORT-B

Write a Java program to implement bubble sort on an array of integers using loops and conditional statements.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class sort
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         int n=sc.nextInt();
8         int[] a=new int[n];
9         System.out.println("enter the element:");
10        for (int i=0;i<n;i++)
11        {
12            a[i]=sc.nextInt();
13        }
14        for(int i=0;i<n-1;i++)
15        {
16            for(int j=0;j<n-i-1;j++){
17                if(a[j]>a[j+1])
18                {
19                    int temp=a[j];
20                    a[j]=a[j+1];
21                    a[i+1]=temp;
```

INPUT

```
5
7 3 5 4 6
```

OUTPUT

```
enter the element:
sorted array:
3 4 5 6 7
```

SORT-I

Write a Java program to implement insertion sort on an array of integers using loops and conditional statements.

```
1 import java.util.Scanner;
2 public class insertionsort
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the number of element:");
8         int n=sc.nextInt();
9         int a[]=new int[n];
10        System.out.println("enter the array element:");
11        for(int i=0;i<n;i++)
12        {
13            a[i]=sc.nextInt();
14        }
15        for(int i=1;i<n;i++)
16        {
17            int temp=a[i];
18            int j;
19            for(j=i-1;j>=0 && a[j]>temp;j--)
20            {
21                a[i+1]=a[i];
```

OUTPUT

```
enter the number of element:
enter the array element:
sorted array
2
4
5
6
8
```

Palindrome



PALINDROME

Write a Java program to check if a given string is a palindrome or not.

```
1 import java.util.Scanner;
2 public class multiplicationtable
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the value of m:");
8         int m=sc.nextInt();
9         System.out.println("enter the value of n:");
10        int n=sc.nextInt();
11        for(int i=1;i<=n;i++)
12        {
13            System.out.println(m+"x"+i+"="+m*i);
14        }
15    }
16 }
```

OUTPUT

```
enter the value of m:
enter the value of n:
4x1=4
4x2=8
4x3=12
4x4=16
4x5=20
```

4

5

Occurrence



OCCURRENCE

Write a Java program to count the number of occurrences of a given character in a string.

PROGRAM EDITOR

Java



Run

Save

```
1 public class pattern
2 {
3     public static void main(String[] args)
4     {
5         for(int i=1;i<=3;i++)
6         {
7             for(int j=1;j<=i;j++)
8             {
9                 System.out.print("1");
10            }
11            System.out.println();
12        }
13        for(int i=3-1;i>=1;i--)
14        {
15            for(int j=1;j<=i;j++){
16                System.out.print("1");
17            }
18            System.out.println();
19        }
20    }
21 }
```

OUTPUT

```
1
11
111
11
1
```

INPUT

Your INPUT goes here!

Check



CHECK

Write a Java program to check if a given string contains only digits.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2 public class amstrong
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc=new Scanner(System.in);
7         System.out.println("enter the number: ");
8         int n=sc.nextInt();
9         int originalnumber=n;
10        int sum=0;
11        while(n>0)
12        {
13            int digit=n%10;
14            sum+=(digit*digit*digit);
15            n/=10;
16        }
17        if(sum==originalnumber)
18        {
19            System.out.println(originalnumber+"is a amstrong number");
20        }
21        else
```

INPUT

153

OUTPUT

```
enter the number:
153is a amstrong number
```

Length ▾

LENGTH

Write a Java program to find the length of the longest substring without repeating characters in a given string.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class composite
2 {
3     public static void main(String[] args)
4     {
5         int[] n={1,2,3,4,5,6,7,8,9,10};
6         int count=0;
7         for(int i=0;i<n.length;i++)
8         {
9             int num=n[i];
10            int factorcount=0;
11            for(int j=1;j<=num;j++)
12            {
13                if(num%j==0){
14                    factorcount++;
15                }
16            }
17            if(factorcount>2)
18            {
19                count++;
20            }
21        }
```

OUTPUT

```
total composite number=5
```

INPUT

Your INPUT goes here!

Lower



LOWER

Write a Java program to convert a given string to lowercase.

PROGRAM EDITOR

Java



Run

Save

```
1 public class lowercase
2 {
3     public static void main(String[] args)
4     {
5         String text="HELLO";
6         String lowertext=text.toLowerCase();
7         String uppertext=text.toUpperCase();
8         System.out.print(lowertext);
9         System.out.print(uppertext);
10    }
11 }
12
```

OUTPUT

```
helloHELLO
```

INPUT

Your INPUT goes here!

Remove



REMOVE

Write a Java program to remove all white spaces from a given string.

PROGRAM EDITOR

Java



Run

Save

```
1 public class main
2 {
3     public static void main(String[] args)
4     {
5         String str=" hello world";
6         String result=str.replaceAll("\\s+","");
7         System.out.println(result);
8     }
9 }
```

INPUT

Your INPUT goes here!

OUTPUT

```
helloworld
```

Reverse



REVERSE

Write a Java program to reverse a given string.

PROGRAM EDITOR

Java



Run

Save

```
1 public class composite
2 {
3     public static void main(String[] args)
4     {
5         int[] n={4,6,8,9,10,12};
6         int count=0;
7         for(int i=0;i<n.length;i++)
8         {
9             int num=n[i];
10            int factorcount=0;
11            int sum=0;
12            for(int j=1;j<=num;j++)
13            {
14                if(num%j==0)
15                {
16                    factorcount++;
17                }
18            }
19            if(factorcount>2)
20            {
21                count++;
```

OUTPUT

6

INPUT

Your INPUT goes here!

Concatenate



CONCATINATE

Write a Java program to concatenate two given strings.

PROGRAM EDITOR

Java



Run

Save

```
1 public class main
2 {
3     public static void main(String[] args)
4     {
5         String s1="java";
6         String s2="programming";
7         String s3=s1.concat("").concat(s2));
8         System.out.println(s3);
9     }
10 }
```

OUTPUT

```
javaprogramming
```

INPUT

Your INPUT goes here!

Repeat



REPEAT

Write a Java program to find the first non-repeated character in a given string.

PROGRAM EDITOR

Java



Run

Save

```
1 public class main
2 {
3     public static void main(String[] args)
4     {
5         String str="java";
6         for(int i=0;i<str.length();i++)
7         {
8             if(str.indexOf(str.charAt(i))==str.lastIndexOf(str.charAt(i)))
9             {
10                 System.out.println("first non repeated character="+str.charAt(i));
11                 break;
12             }
13         }
14     }
15 }
```

OUTPUT

```
first non repeated character=j
```

INPUT

Your INPUT goes here!

Frequent ▼

FREQUENT

Write a Java program to find the most frequent word in a given string.

PROGRAM EDITOR

Java ▼

Run

Save

```
1 import java.util.*;
2
3 public class Main
4 {
5     public static void main(String[] args)
6     {
7         Scanner sc = new Scanner(System.in);
8         String input = sc.nextLine();
9
10        String[] words = input.trim().isEmpty() ? new String[0] : input.toLowerCase().
11
12        Map<String, Integer> freq = new HashMap<>();
13        for (String w : words) {
14            if (!w.isEmpty()) freq.put(w, freq.getOrDefault(w, 0) + 1);
15        }
16
17        String mostFrequent = "";
18        int max = 0;
19
20        for (Map.Entry<String, Integer> e : freq.entrySet())
21
```

OUTPUT

is

INPUT

java is a
programming
language and it is
easy

Substring

SUBSTRING

Write a Java program to create a substring from a given string.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.*;  
2  
3 public class Main {  
4     public static void main(String[] args) {  
5         Scanner sc = new Scanner(System.in);  
6  
7         System.out.println("INPUT:");  
8         System.out.print("Enter a string: ");  
9         String s = sc.nextLine();  
10  
11        System.out.print("Enter start index: ");  
12        int start = sc.nextInt();  
13  
14        System.out.print("Enter end index: ");  
15        int end = sc.nextInt();  
16  
17        if (start < 0 || end > s.length() || start > end) {  
18            System.out.println("Invalid input");  
19        } else {  
20            String sub = s.substring(start, end);  
21            System.out.println("OUTPUT:");
```

OUTPUT

```
INPUT:  
Enter a string: Enter start index: Enter end index: OUTPUT:  
jav
```

INPUT

```
java  
0  
3
```

Compare

COMPARE

Write a Java program to compare two strings lexicographically.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         String s1 = sc.nextLine();
8         String s2 = sc.nextLine();
9
10        int result = s1.compareTo(s2);
11
12        if (result == 0) {
13            System.out.print("Both strings are equal");
14        } else if (result < 0) {
15            System.out.print("First string is lexicographically smaller");
16        } else {
17            System.out.print("Second string is lexicographically smaller");
18        }
19    }
20 }
```

OUTPUT

```
First string is lexicographically smaller
```

INPUT

```
java
programming
```

Suffix



SUFFIX

Write a Java program to check if a given string ends with a specific suffix.

PROGRAM EDITOR

Java



Run

Save

```
1 public class EndsWithExample
2 {
3     public static void main(String[] args)
4     {
5
6         String str = "programming";
7         String suffix = "ing";
8
9         if (str.endsWith(suffix)) {
10             System.out.println("The string ends with \"" + suffix + "\".");
11         } else {
12             System.out.println("The string does not end with \"" + suffix + "\".");
13         }
14     }
15 }
16
```

OUTPUT

```
The string ends with "ing".
```

INPUT

Your INPUT goes here!



FIND

Write a Java program to find the index of a given character or substring in a given string.

PROGRAM EDITOR

Java 

Run

Save

```
1 public class main
2 {
3     public static void main(String[] args)
4     {
5         String str="hello world";
6         System.out.println("index of'o':" +str.indexOf('o'));
7     }
8 }
```

OUTPUT

```
index of'o':4
```

INPUT

Your INPUT goes here!

QUESTIONS 20 / 20 completed

Upper



UPPER

Write a Java program to convert a given string to uppercase.

PROGRAM EDITOR

Java



Run

Save

```
1 public class main
2 {
3     public static void main(String[] args)
4     {
5         String s="java";
6         System.out.print("upper case="+s.toUpperCase());
7     }
8 }
```

OUTPUT

```
upper case=JAVA
```

INPUT

Your INPUT goes here!

RM-Char



RM-CHAR

Write a Java program to remove a specified character from a given string.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         String str = sc.nextLine();
7         char ch = sc.nextLine().charAt(0);
8
9         String result = str.replace(String.valueOf(ch), "");
10        System.out.println(result);
11    }
12 }
```

OUTPUT

```
jv proqramming
```

INPUT

```
java programming
a
```

Replace



REPLACE

Write a Java program to replace a specified character with another character in a given string.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         String s = sc.nextLine();
8         char oldCh = sc.nextLine().charAt(0);
9         char newCh = sc.nextLine().charAt(0);
10
11         String result = s.replace(oldCh, newCh);
12         System.out.println(result);
13
14         sc.close();
15     }
16 }
```

OUTPUT

```
java progranning
```

INPUT

```
java programming
m
n
```


Split



SPLIT

Write a Java program to split a given string into an array of substrings based on a specified delimiter.

PROGRAM EDITOR

Java



Run

Save

```

1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         String str = sc.nextLine();
8         String delimiter = sc.nextLine();
9
10        if (delimiter.length() == 0) {
11            for (int i = 0; i < str.length(); i++) {
12                System.out.println(str.charAt(i));
13            }
14            return;
15        }
16
17        String[] parts = str.split(java.util.regex.Pattern.quote(delimiter), -1);
18        for (String part : parts) {
19            System.out.println(part);
20        }
21    }

```

OUTPUT

Exception in thread "main" java.util.NoSuchElementException: No line found

INPUT

Your INPUT goes here!

Trim



TRIM

Write a Java program to trim a given string and remove leading and trailing spaces.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.Scanner;  
2 public class main  
3 {  
4     public static void main(String[] args)  
5     {  
6         Scanner sc=new Scanner(System.in);  
7         System.out.println("enter the string");  
8         String str=sc.nextLine();  
9         String trimmed=str.trim();  
10        System.out.print("Trimmed String:"+trimmed);  
11    }  
12 }
```

OUTPUT

```
enter the string  
Trimmed String:hello world
```

INPUT

hello world

Convert ▼

CONVERT

Write a Java program to convert a given string to a char array.

PROGRAM EDITOR

Java ▼

Run

Save

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         String str = sc.hasNextLine() ? sc.nextLine() : "";
7         char[] arr = str.toCharArray();
8
9         for (int i = 0; i < arr.length; i++) {
10             System.out.print(arr[i]);
11             if (i < arr.length - 1) System.out.print(" ");
12         }
13     }
14 }
```

OUTPUT

INPUT

Your INPUT goes here!

Logger

LOGGER

You are working on a project that requires the use of a logging framework. Write a Java program to create an interface called "Logger" with methods for logging different types of messages, such as "debug", "info", "warning", and "error". Implement the interface in a class called "Log4jLogger"

PROGRAM EDITOR

Java



Run

Save

```

1 import java.util.*;
2 public class Alist
3 {
4     public static void main(String[] args)
5     {
6         ArrayList<String>student=new ArrayList<String>();
7         student.add("Ram");
8         student.add("Loki");
9         student.add("Siva");
10        System.out.println(student);
11        student.add(0,"Khan");
12        System.out.println(student);
13        System.out.println(student.size());
14        student.set(3,"Loki");
15        System.out.println(student);
16        System.out.println(student.contains("Ram"));
17    }
18 }
    
```

OUTPUT

```

[Ram, Loki, Siva]
[Khan, Ram, Loki, Siva]
4
[Khan, Ram, Loki, Loki]
true
    
```

INPUT

Your INPUT goes here!

Character

CHARACTER

You are developing a game that involves different types of characters, such as heroes, villains, and monsters. Write a Java program to create an interface called "Character" with methods for moving, attacking, and taking damage. Implement the interface in classes for each type of character, such as

```
1 public class Main {
2     public static void main(String[] args) {
3         Character hero = new Hero(100);
4         Character villain = new Villain(120);
5         Character monster = new Monster(150);
6
7         hero.move();
8         hero.attack();
9         hero.takeDamage(30);
10
11        villain.move();
12        villain.attack();
13        villain.takeDamage(40);
14
15        monster.move();
16        monster.attack();
17        monster.takeDamage(50);
18    }
19 }
20 interface Character {
21     void move();
```

OUTPUT

```
Hero is moving.
Hero attacks!
Hero took 30 damage. Health: 70
Villain is moving.
Villain attacks!
Villain took 40 damage. Health: 80
Monster is moving.
Monster attacks!
Monster took 50 damage. Health: 100
```

Your INPUT goes
here!

QUESTIONS 8 / 8 completed

Sort

SORT

Write a Java program to create a list of integers and sort them in ascending order using the Collections framework.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.*;
2 public class main
3 {
4     public static void main(String[] args)
5     {
6         List<Integer>numbers=new ArrayList<>();
7         numbers.add(50);
8         numbers.add(10);
9         numbers.add(30);
10        numbers.add(40);
11        numbers.add(20);
12        Collections.sort(numbers);
13        System.out.println("Sorted list:"+numbers);
14    }
15 }
```

OUTPUT

```
Sorted list:[10, 20, 30, 40, 50]
```

INPUT

Your INPUT goes here!

HashSet



HASHSET

Write a Java program to create a HashSet of strings and remove a specific element from it.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.*;
2 public class alist
3 {
4     public static void main(String[] args)
5     {
6         Map<Integer,String>students=new HashMap<Integer,String>();
7         students.put(101,"Ram");
8         students.put(102,"Loki");
9         students.put(103,"Siva");
10        System.out.println(students);
11        System.out.println(students.get(103));
12        System.out.println(students.entrySet());
13    }
14 }
```

OUTPUT

```
{101=Ram, 102=Loki, 103=Siva}
Siva
[101=Ram, 102=Loki, 103=Siva]
```

INPUT

Your INPUT goes here!

TreeMap



TREEMAP

Write a Java program to create a TreeMap of strings and integers and iterate over its entries.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.*;
2
3 public class TreeMapExample {
4     public static void main(String[] args) {
5         TreeMap<String, Integer> map = new TreeMap<>();
6
7         map.put("Apple", 50);
8         map.put("Banana", 30);
9         map.put("Orange", 20);
10        map.put("Kiwi", 40);
11        map.put("Mango", 10);
12
13        for (Map.Entry<String, Integer> entry : map.entrySet()) {
14            System.out.println("Key: " + entry.getKey() + ", Value: " + entry.getValue());
15        }
16    }
17 }
```

OUTPUT

```
Key: Apple, Value: 50
Key: Banana, Value: 30
Key: Kiwi, Value: 40
Key: Mango, Value: 10
Key: Orange, Value: 20
```

INPUT

Your INPUT goes here!

PRIORITYQUEUE

Write a Java program to create a PriorityQueue of integers and add elements to it.

```
35     System.out.println("After Poll: " + dq);
36     System.out.println("Contains Apple? " + dq.contains("Apple"));
37     System.out.println("Size: " + dq.size());
38     System.out.println("Is Empty? " + dq.isEmpty());
39     System.out.print("Forward: ");
40     for (String fruit : dq)
41     {
42         System.out.print(fruit + " ");
43     }
44     System.out.print("\nReverse: ");
45     var it = dq.descendingIterator();
46     while (it.hasNext())
47     {
48         System.out.print(it.next() + " ");
49     }
50     dq.clear();
51     System.out.println("\nAfter Clear: " + dq);
52     System.out.println("Is Empty? " + dq.isEmpty());
53 }
54 }
55
```

OUTPUT

```
Deque: [Orange, Apple, Banana, Mango, Grapes]
First Element: Orange
Last Element: Grapes
Peek First: Orange
Peek Last: Grapes
Remove First: Orange
Remove Last: Grapes
After Removal: [Apple, Banana, Mango]
After offerFirst & offerLast: [Pineapple, Apple, Banana, Mango, Watermelon]
Poll First: Pineapple
```

LinkedListHashMap ▾

LINKEDHASHMAP

Write a Java program to create a LinkedHashMap of strings and integers and retrieve the keys in the order they were inserted.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 import java.util.LinkedHashMap;
2 import java.util.Map;
3
4 public class Main {
5     public static void main(String[] args) {
6         LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
7         map.put("Apple", 1);
8         map.put("Banana", 2);
9         map.put("Cherry", 3);
10        map.put("Date", 4);
11
12        for (Map.Entry<String, Integer> entry : map.entrySet()) {
13            System.out.println(entry.getKey() + " = " + entry.getValue());
14        }
15    }
16 }
```

OUTPUT

```
Apple = 1
Banana = 2
Cherry = 3
Date = 4
```

INPUT

Your INPUT goes here!

Max-Min



MAX-MIN

Write a Java program to create a Set of integers and perform different operations on it, such as finding the maximum and minimum values.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.*;  
2 public class main  
3 {  
4     public static void main(String[] args)  
5     {  
6         Set<Integer>value=new HashSet<Integer>();  
7         value.add(10);  
8         value.add(5);  
9         value.add(15);  
10        value.add(7);  
11        value.add(36);  
12        int max=Integer.MIN_VALUE;  
13        int min=Integer.MAX_VALUE;  
14        for(int n: value)  
15        {  
16            if(n>max)  
17                max=n;  
18            else if(n<min)  
19                min=n;  
20        }  
21        System.out.println("value:"+value);
```

OUTPUT

```
value:[36, 5, 7, 10, 15]  
maximum value=36  
minimum value=5
```

INPUT

Your INPUT goes here!

1-10 T

1-10 T

Write a Java program to create a simple thread that prints the numbers from 1 to 10.

PROGRAM EDITOR

Java



Run

Save

```
1 public class main
2 {
3     public static void main(String[] args)
4     {
5         try
6         {
7             int a=10/5;
8             String name="raj";
9             System.out.println(name.charAt(0));
10            System.out.println(name.length());
11            System.out.print(a);
12        }
13        catch (StringIndexOutOfBoundsException e)
14        {
15            System.out.println("zero division error occured");
16        }
17        finally
18        {
19            System.out.println("this is always executed");
20        }
21    }
```

OUTPUT

```
r
3
2this is always executed
```

INPUT

Your INPUT goes here!

Counter-T ▾

COUNTER-T

Write a Java program to create a thread-safe counter that can be accessed by multiple threads simultaneously.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class main
2 {
3     public static void main(String[] args)
4     {
5         Object[] a=new String[3];
6         try
7         {
8             a[0]=10;
9             System.out.println(a[0]);
10        }
11        catch(ArrayStoreException e)
12        {
13            System.out.println("array store exception accured");
14        }
15    }
16 }
17
```

OUTPUT

```
array store exception accured
```

INPUT

Your INPUT goes here!

M-Thread



M-THREAD

Write a Java program to create multiple threads that perform some time-consuming task, such as calculating prime numbers or sorting an array, and compare their execution times.

PROGRAM EDITOR

Java



Run

Save

```
1 class MyThread extends Thread
2 {
3     public void run()
4     {
5         for(int i=1;i<=5;i++)
6         {
7             System.out.println(getName() + ":" + i);
8         }
9     }
10    public static void main(String[] args)
11    {
12        MyThread t1=new MyThread();
13        MyThread t2=new MyThread();
14        t1.setName("Thread 1");
15        t2.setName("Thread 2");
16        t1.start();
17        t2.start();
18    }
19 }
```

OUTPUT

```
Thread 1:1
Thread 1:2
Thread 2:1
Thread 1:3
Thread 2:2
Thread 2:3
```

INPUT

Your INPL
here!

Pro-Consume



PRO-CONSUME

Write a Java program to create a producer-consumer scenario using multiple threads. The producer thread should add items to a queue, and the consumer thread should remove items from the queue and process them.

PROGRAM EDITOR

Java



Run

Save

```
1 class A
2 {
3     A()
4     {
5         System.out.println("costructor A");
6     }
7     void A()
8     {
9         System.out.println("method A");
10    }
11    public static void main(String[] args)
12    {
13        System.out.println("main mathod");
14        A a=new A();
15        System.out.println("after object created");
16        a.A();
17    }
18 }
```

OUTPUT

```
main mathod
costructor A
after object created
method A
```

INPUT

Your INPUT
here!

Deadlock



DEADLOCK

Write a Java program to create a deadlock scenario using multiple threads.

PROGRAM EDITOR

Java



Run

Save

```
1 class Main {
2
3     public static void main(String[] args) {
4
5         Shared obj = new Shared();
6
7         Thread t1 = new Thread(() -> {
8             obj.waitThread();
9         });
10
11        Thread t2 = new Thread(() -> {
12            try {
13                Thread.sleep(1000);
14            } catch (InterruptedException e) {
15                System.out.println("Thread 2 interrupted");
16            }
17
18            obj.notifyThread();
19        });
20
21        t1.start();
```

OUTPUT

```
Thread 1 is waiting
Thread 2 is notifying
Thread 1 is resumed
```

INPUT

Your INPUT goes
here!

Wait-Notify



WAIT-NOTIFY

Write a Java program to demonstrate how to use the "wait" and "notify" methods to implement a producer-consumer scenario.

PROGRAM EDITOR

Java



Run

Save

```
1 class main
2 {
3     public static void main(String[] args) throws InterruptedException
4     {
5         MyThread1 t1=new MyThread1();
6         t1.start();
7         Thread.sleep(100);
8         synchronized(t1)
9         {
10             System.out.println("main method is notifying thread 1");
11             t1.notify();
12         }
13     }
14 }
15 class MyThread1 extends Thread
16 {
17     public void run()
18     {
19         synchronized(this)
20         {
21             try
```

OUTPUT

```
Thread 1 is waiting
main method is notifying thread 1
thread 1 is resumed
```

INPUT

Your INPUT
here!

Pool



POOL

Write a Java program to create a thread pool that can execute multiple threads concurrently.

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3 public class ThreadPoolExample {
4     public static void main(String[] args) {
5         ExecutorService executor = Executors.newFixedThreadPool(5);
6         for (int i = 1; i <= 10; i++) {
7             int taskId = i;
8             executor.execute(() -> {
9                 System.out.println("Task " + taskId + " executed by " + Thread.currentThread().getName());
10            });
11        }
12        executor.shutdown();
13    }
14 }
```

OUTPUT

```
Task 2 executed by pool-1-thread-2
Task 3 executed by pool-1-thread-3
Task 5 executed by pool-1-thread-5
Task 4 executed by pool-1-thread-4
Task 1 executed by pool-1-thread-1
Task 9 executed by pool-1-thread-4
```

INPUT

Your INPUT
here!

JOIN

Write a Java program to demonstrate how to use the "join" method to wait for a thread to complete its execution before proceeding with the main thread.

```
1 public class MyThread {
2     public static void main(String[] args) {
3         WorkerThread t = new WorkerThread();
4         t.start();
5         try {
6             t.join();
7         } catch (InterruptedException e) {
8             System.out.println(e);
9         }
10        System.out.println("Main thread continues after child thread completes.");
11    }
12 }
13 class WorkerThread extends Thread {
14     public void run() {
15         for (int i = 1; i <= 5; i++) {
16             System.out.println("Child Thread: " + i);
17             try {
18                 Thread.sleep(500);
19             } catch (InterruptedException e) {
20                 System.out.println(e);
21             }
22         }
23     }
24 }
```

OUTPUT

```
Child Thread: 1
Child Thread: 2
Child Thread: 3
Child Thread: 4
Child Thread: 5
Child thread completed.
Main thread continues after child thread completes.
```

SAHRED

Write a Java program to demonstrate how to use synchronization to prevent multiple threads from accessing a shared resource simultaneously.

```
1 class SharedResource {
2
3     public synchronized void useResource(String threadName) {
4         System.out.println(threadName + " entered the resource.");
5         System.out.println(threadName + " is using the resource.");
6         System.out.println(threadName + " leaving the resource.");
7     }
8
9     public static void main(String[] args) {
10
11         SharedResource resource = new SharedResource();
12
13         Thread t1 = new Thread(() -> {
14             resource.useResource("Thread-1");
15         });
16
17         Thread t2 = new Thread(() -> {
18             resource.useResource("Thread-2");
19         });
20
21         Thread t3 = new Thread(() -> {
```

OUTPUT

```
Thread-1 entered the resource.
Thread-1 is using the resource.
Thread-1 leaving the resource.
Thread-2 entered the resource.
Thread-2 is using the resource.
Thread-2 leaving the resource.
Thread-3 entered the resource.
Thread-3 is using the resource.
Thread-3 leaving the resource.
```

AtomicInteger

ATOMICINTEGER

Write a Java program to demonstrate how to use the AtomicInteger class to provide atomic access to a shared integer variable.

Output:

Shared value: 20000

This confirms that the two threads were able to increment the shared value atomically and the final value of the shared value is as

PROGRAM EDITOR

Java



Run

Save

```
1 import java.util.concurrent.atomic.AtomicInteger;
2 public class Main
3 {
4     public static void main(String[] args) throws InterruptedException
5     {
6         AtomicInteger sharedValue = new AtomicInteger(0);
7
8         Runnable task = () -> {
9             for (int i = 0; i < 10000; i++)
10             {
11                 sharedValue.incrementAndGet();
12             }
13         };
14         Thread t1 = new Thread(task);
15         Thread t2 = new Thread(task);
16         t1.start();
17         t2.start();
18         t1.join();
19         t2.join();
20
21         System.out.println("Shared value: " + sharedValue.get());
```

OUTPUT

Shared value: 20000

INPUT

Your INPUT
here!

Employee

EMPLOYEE

Write a Java program to implement a basic employee management system using inheritance. Create a parent class called Employee with attributes such as name, id, and salary. Then, create child classes such as Manager and SalesPerson that inherit from the Employee class and have their own unique methods for

```
1 public class EmployeeManagementSystem {
2     public static void main(String[] args) {
3         Manager manager = new Manager(
4             "John", 101, 75000, 5);
5         SalesPerson salesperson = new SalesPerson(
6             "Alice", 102, 50000, 10000);
7
8         System.out.println("=== Manager Details ===");
9         manager.displayDetails();
10        manager.manageTeam();
11
12        System.out.println();
13
14        System.out.println("=== SalesPerson Details ===");
15        salesperson.displayDetails();
16        salesperson.makeSale(2500);
17    }
18 }
19 class Employee
20 {
21     protected String name;
```

OUTPUT

```
=== Manager Details ===
Name: John
ID: 101
Salary: $75000.0
Team Size: 5
John is managing a team of 5 employees.

=== SalesPerson Details ===
Name: Alice
ID: 102
Salary: $50000.0
```

Your INPUT c
here!

ANIMAL-1

Write a Java program to implement a basic animal classification system using inheritance. Create a parent class called Animal with attributes such as name and habitat. Then, create child classes such as Mammal, Reptile, and Bird that inherit from the Animal class and have their own unique methods for eating and reproducing.

```
1 public class Main {
2     public static void main(String[] args) {
3         Mammal mammal = new Mammal("Lion", "Grassland");
4         Reptile reptile = new Reptile("Snake", "Forest");
5         Bird bird = new Bird("Eagle", "Mountains");
6         Animal[] animals = { mammal, reptile, bird };
7         for (Animal a : animals) {
8             System.out.println("Name: " + a.name + ", Habitat: " + a.habitat);
9             a.eat();
10            a.reproduce();
11            System.out.println();
12        }
13    }
14 }
15 class Animal {
16     String name;
17     String habitat;
18     Animal(String name, String habitat) {
19         this.name = name;
20         this.habitat = habitat;
21     }
22 }
```

OUTPUT

```
Name: Lion, Habitat: Grassland
Lion (Mammal) eats meat/plants and feeds on milk as a baby.
Lion (Mammal) gives birth to live young ones.

Name: Snake, Habitat: Forest
Snake (Reptile) eats insects or small animals.
Snake (Reptile) lays eggs on land.

Name: Eagle, Habitat: Mountains
Eagle (Bird) eats seeds, insects, or small animals.
```

Your INPUT goes here!